

第 7 章：梯度下降

第 2 节：梯度下降优化 (Gradient Descent Optimization)

《深度学习基础》课程小组

2026 年 10 月 7 日

目录

- 1 迭代数值方法
- 2 梯度信息的价值
- 3 批量梯度下降
- 4 随机梯度下降
- 5 小批量
- 6 参数初始化
- 7 总结
- 8 参考文献

1. 为什么需要迭代数值方法？

- 对于神经网络定义的复杂误差函数，几乎不可能找到 $\nabla E(\mathbf{w}) = 0$ 的解析解。
- 因此需要迭代数值方法。
- 大多数方法：选取初始值 $\mathbf{w}^{(0)}$ ，然后逐步移动：

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta \mathbf{w}^{(\tau-1)} \quad (7.15)$$

- 不同算法的区别在于更新量 $\Delta \mathbf{w}^{(\tau)}$ 的选择。

1.1 初始值的重要性

- 误差曲面形状复杂，找到的解**取决于初始参数 $w^{(0)}$** 。
- 为了找到足够好的解：
 - 可能需要**多次运行**基于梯度的算法。
 - 每次使用**不同的随机初始点**。
 - 在独立的**验证集**上比较结果性能。

2. 为什么梯度信息如此重要？

- 深度神经网络的梯度可用**误差反向传播**高效计算。
- 利用梯度信息能**显著提升训练速度**。

信息效率的对比

- 不用梯度**：每次函数求值只获得 **1 个标量信息**。
- 用梯度**：每次梯度计算获得 **W 个独立信息** (W 维向量)。

2.1 计算复杂度分析

误差函数的二次近似由 $\mathbf{b}$ (W 维) 和 $\mathbf{H}$ ($W \times W$ 对称矩阵) 确定, 共 $W(W+3)/2$ 个独立元素。

	不用梯度	用梯度 (反向传播)
每次计算获得信息量	1 个 (标量)	W 个 (梯度向量)
所需计算次数	$\mathcal{O}(W^2)$	$\mathcal{O}(W)$
每次计算代价	$\mathcal{O}(W)$	$\mathcal{O}(W)$
总计算量	$\mathcal{O}(W^3)$	$\mathcal{O}(W^2)$

- 梯度方法比不用梯度快一个数量级。
- 这是所有实用神经网络训练算法的基础。

3. 批量梯度下降 (Batch Gradient Descent)

- 最简单的梯度方法：沿负梯度方向走一小步：

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E(\mathbf{w}^{(\tau-1)}) \quad (7.16)$$

- $\eta > 0$ 称为学习率 (learning rate)。
- 每一步都沿误差下降最快的方向移动，故又称最速下降法。
- 误差函数定义在整个训练集上，每步需处理全部数据。
- 一次使用全部数据的方法称为批方法 (batch methods)。

4. 随机梯度下降 (SGD)

- 深度学习方法受益于**超大规模数据集**。
- 但批方法在数据量大时**极其低效**——每次评估梯度都要处理全部数据。
- 基于最大似然的误差函数是**各项之和**:

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}) \quad (7.17)$$

- SGD 每次只基于**一个数据点**更新权重:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E_n(\mathbf{w}^{(\tau-1)}) \quad (7.18)$$

4.1 SGD 的关键术语 I

训练轮次 (Training Epoch)

完整遍历一次整个训练集。

在线梯度下降 (Online Gradient Descent)

当数据来源于连续的新数据流时，SGD 的别称。

4.1 SGD 的关键术语 II

Algorithm 1 Stochastic gradient descent

Input: Training set $\{1, \dots, N\}$; error $E_n(\mathbf{w})$; learning rate η ; initial $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$

repeat

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_n(\mathbf{w})$

// update weight

$n \leftarrow n + 1 \pmod{N}$

// iterate over data

until convergence

return $\mathbf{w}$

4.2 SGD 的优势：处理冗余数据

- 考虑极端例子：将数据集**复制加倍**。
- 数学上等价于将误差函数乘以 2（调整学习率即可补偿）。
- **批量方法**：需要**双倍计算量**来评估梯度。
- **SGD**：**不受影响**——单步仍只处理 1 个样本。

另一个优势：逃离局部极小值

对整个数据集的驻点，通常**不是**单个数据点的驻点，因此 SGD 有可能跳出局部极小值。

5. 小批量 (Mini-batches)

- SGD 的缺点：单点梯度是噪声极大的估计。
- 折中方案：使用一小部分数据点（小批量）评估梯度。
- 批量大小的选择：
 - 基于 N 个样本计算均值的误差为 $\sigma/\sqrt{N}$ 。
 - 增大批量，收益递减：增大 100 倍，误差仅缩小 10 倍。
 - 考虑硬件效率：2 的幂次（64, 128, 256, ...）通常性能较好。

5.1 小批量的数据选取 I

- 数据点应**随机选取**，因为原始数据可能存在相关性（如按字母或日期排序）。
- 常用做法：**随机打乱**整个数据集，然后依次抽取连续数据块。
- 各轮迭代之间可**重新打乱**，帮助逃离局部极小值。
- 即使使用小批量，该算法通常仍被称为“随机梯度下降”。

5.1 小批量的数据选取 II

Algorithm 2 Mini-batch stochastic gradient descent

Input: Training set $\{1, \dots, N\}$; batch size B ; error $E_{n:n+B-1}(\mathbf{w})$; η ; initial $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$

repeat

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{n:n+B-1}(\mathbf{w})$

// update weight

$n \leftarrow n + B$

// next mini-batch

if $n > N$ **then**

shuffle data; $n \leftarrow 1$

end if

until convergence

return $\mathbf{w}$

6. 参数初始化 (Parameter Initialization)

- 迭代算法需要初始参数设置。
- 初始化方式显著影响收敛速度和泛化性能。
- 目前理论指导相对较少。

关键考量：对称性破缺 (Symmetry Breaking)

- 若同一层所有单元参数初始化为相同值 (如全 0):
 - 前向传播：输出完全相同。
 - 反向传播：梯度完全相同。
 - 更新后：权重仍然完全相同 → 完全冗余。
- 解决方案：从某种分布中随机初始化，打破对称性。

6.1 初始化分布与 He 初始化

- 常用分布：均匀分布 $[-\epsilon, \epsilon]$ 或零均值高斯 $\mathcal{N}(0, \epsilon^2)$ 。
- ϵ 的选择至关重要。

He 初始化 (He et al., 2015b)

考虑第 ℓ 层：

$$a_i^{(\ell)} = \sum_{j=1}^M w_{ij} z_j^{(\ell-1)}, \quad z_i^{(\ell)} = \text{ReLU}(a_i^{(\ell)}).$$

若权重来自 $\mathcal{N}(0, \epsilon^2)$, $z_j^{(\ell-1)}$ 方差为 λ^2 , 则：

$$\mathbb{E}[a_i^{(\ell)}] = 0, \quad \text{var}[z_j^{(\ell)}] = \frac{M}{2} \epsilon^2 \lambda^2.$$

要求逐层方差稳定 $\Rightarrow \epsilon = \sqrt{2/M}$ (7.23)

6.2 He 初始化中的“2”从何而来？

- ReLU 将一半负数输出截断为 0。
- 数据方差每经过一次 ReLU 损失一半 (公式 7.22 中的 $1/2$)。
- 为弥补损失, He 初始化在标准差中乘上 $\sqrt{2}$ (公式 7.23)。
- 目的: 让信号在深层网络中传播时方差保持稳定。

前提条件

公式 (7.21)(7.22) 的推导需要 w_{ij} 与 $z_j^{(\ell-1)}$ 相互独立, 且均值都为零。

6.3 其他初始化考虑

- 可将初始化尺度 ϵ 视为**超参数**，多次训练尝试不同取值。
- **偏置初始化**：通常设为**小正值**，确保学习初期大多数激活前处于激活状态。
 - 对 ReLU 单元尤为重要——希望激活前为正，产生非零梯度。
- **迁移学习初始化**：利用其他任务训练所得参数，或各种无监督训练得到的参数。

什么是超参数 (Hyperparameter)?

训练开始前由人类设定的参数，训练过程中保持不变。

例如：学习率 η 、初始化尺度 ϵ 、小批量大小、网络结构、正则化系数。

7. 本节总结

- 神经网络优化依赖**迭代数值方法**，无法求解析解。
- **梯度信息**至关重要：信息效率高 W 倍，总计算量从 $\mathcal{O}(W^3)$ 降到 $\mathcal{O}(W^2)$ 。
- **批量梯度下降**：每步使用全部数据，沿负梯度方向更新。
- **随机梯度下降 (SGD)**：每次用一个样本，高效处理大数据和冗余。
- **小批量 SGD**：折中方案，兼顾噪声与效率，需随机打乱数据。
- **参数初始化**：
 - 必须**随机初始化**以打破对称性。
 - **He 初始化**： $\epsilon = \sqrt{2/M}$ ，补偿 ReLU 带来的方差衰减。
 - 偏置通常初始化为**小正值**。

8. 参考文献

- 教材: Bishop, C. M. & Bishop, H. (2023). Deep Learning Foundations and Concepts. Springer Nature Switzerland AG.
- Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification.